

This sheet provides some common risks in student projects. You should check if it applies to your group project. You should also feel free to add other risks. Exemplary analysis is also added.

Risk Category	Risk Title	Description	#	Estimated likelihood of occurrence (1:1-5 with 1 lowest)	Estimated Impact (1:1-5 with 1 lowest impact)
Personel	Loss team members	One or more team members drop the class		1	4
	add new team members	A new member joins the team mid-project and needs to ramp up.		2	2
	Lack of motivation or responsibility	A member disengages or misses assigned tasks.		2	3
Communication	duplicate work	Two members unknowingly build the same feature.		3	2
	worked on wrong components	A member builds a component that doesn't match agreed requirements.		2	3
	useless work	Work is done on something that turns out not to be needed.		2	2
	inconsistent work	Code across the app is written with inconsistent style/assumptions.		3	2
Requirements	unclear requirements	A user story is ambiguous, leading to rework.		3	3
	scope creep	New requirements are added mid-iteration beyond what was committed.		3	3

Management	constant requirements changes	Requirements change repeatedly across iterations, causing rework.	2	3
	improper task assignments	Tasks are assigned to members without the right skills/bandwidth.	2	3
	improper planning	Sprint plans underestimate effort or dependencies.	2	3
	lack of management s	The iteration's team lead lacks experience coordinating the team.	2	2
Technology competence	Not familiar with the framework used	A member is unfamiliar with React/Express or another core framework.	3	2
	Not familiar with the programming language used	A member is new to JavaScript/Node.js.	2	2
	Not familiar with unit testing	Members lack experience writing unit tests.	3	3
	Other technology incompetence	A gap with another required tool (Docker, CI/CD, hosting) slows the team.	2	2
	Not familiar with Git	A member is inexperienced with Git/GitHub workflows.	3	2
Design and implementation	Improper design	The architecture or data model doesn't hold up under real requirements.	2	4

	improper technology stack	A chosen library/tool in the stack turns out to be a poor fit.	2	3
	Messy code	Inconsistent or poorly structured code accumulates over time.	3	2
Testing	Not enough testing	Limited time/experience leads to insufficient automated test coverage.	3	4
Integration and deployment	Not enough time for integration and deployment	Integration/deployment is left too late, risking a rushed final demo.	3	4
		Local, staging and production differ (Node version, environment variables, database tier), so code that passes locally fails once deployed — or a deployment-only bug never reproduces on a developer machine.	3	3

Integration and deployment dev/prod parity

One member owns most of the infrastructure (CI workflow, Render and Vercel deployment, the staging mirror repository and its secrets). If that person is unavailable, the team cannot deploy or fix a broken pipeline.

3

4

Management

key-person
dependency

The application's reachable addresses belong to third-party platforms (*.onrender.com, *.vercel.app, MongoDB Atlas). A hostname change, a free-tier suspension or an idle-instance cold start breaks the demo, and the team does not control any of it.

3

3

**Integration and
deployment**

external hostname
dependency

Design and implementation	Unbounded anonymous tenant creation (SCRUM-114)	POST /api/organizations is public by necessity — it bootstraps a new organization and its first admin — but it was uncapped. Reproduced: 12 anonymous requests created 12 organizations and 12 admin accounts, each costing roughly 330 ms of bcrypt work on the server.	4	4
		SDD §6.6 promises audit-log immutability at two layers. The application layer is done, but the API's database user still holds a role that could update or delete audit rows. Anything that bypasses Mongoose — a raw driver call, or a successful injection — is therefore not stopped by anything.	2	4
Design and implementation	Audit log immutability not enforced at the database layer (SCRUM-125)			

#	Estimated retirement Cost (C: 1-5 with 1 lowest cost)	Priority (lowest number has highest priority)	Responsible engineer	Target completion date	Detailed plan
	4	4*2*4=32	team leader and all members	11/01/2026	Redefine the project scope if necessary, and assign the tasks to other team members.
	2	4*4*2=32	Team Lead	Ongoing	Keep onboarding docs (setup guide, coding standards) current; assign an onboarding buddy.
	3	4*3*3=36	Team Lead	Ongoing	Track task status on the team board; team lead follows up early and reassigns if needed.
	2	3*4*2=24	Team Lead	Ongoing	Assign tasks explicitly on the sprint board; post a note in team chat before starting work.
	2	4*3*2=24	Architect / Lead Dev	Ongoing	Document acceptance criteria in the ticket before implementation starts; owner confirms scope first.
	2	4*4*2=32	Team Lead	Ongoing	Confirm each task against the current sprint backlog before starting.
	2	3*4*2=24	Lead Developer	Ongoing	Adopt a shared ESLint/Prettier config; require one peer review before merging.
	2	3*3*2=18	Requirements Engineer (rotating)	Ongoing	Require acceptance criteria on every story before it enters a sprint.
	2	3*3*2=18	Team Lead	Ongoing	Freeze scope at sprint start; log new requests to the backlog for a future iteration.

3	$4*3*3=36$	Team Lead / Req. Engineer	Ongoing	Version the requirements doc; approve changes only at iteration boundaries.
2	$4*3*2=24$	Team Lead	Ongoing	Match tasks to skills during sprint planning; check in mid-sprint on progress.
2	$4*3*2=24$	Team Lead	Ongoing	Break stories into small tasks; size sprints using prior iteration's actual velocity.
2	$4*4*2=32$	Current iteration's Team Lead	Ongoing	Use a repeatable process (standups, shared board, retros); outgoing lead hands off notes.
2	$3*4*2=24$	Most experienced dev on that layer	First 1-2 sprints	Pair unfamiliar members with an experienced teammate for their first tasks.
2	$4*4*2=32$	Team member's mentor	First 1-2 sprints	Assign smaller, well-scoped tasks first; use linting to catch mistakes early.
2	$3*3*2=18$	QA / Test Lead	Ongoing	QA lead shares a short testing how-to and one example suite in Sprint 1.
2	$4*4*2=32$	DevOps / Integration Lead	Ongoing	Run a quick team skills survey; allocate setup time in Sprint 1 for gaps found.
1	$3*4*1=12$	Team Lead	Ongoing	Agree on one branching strategy (feature branches + PRs); walk new members through it early so everyone's setup matches.
3	$4*2*3=24$	Architect	Reviewed each iteration	Review the data model/API design as a team before implementation starts each iteration.

3	$4 \times 3 \times 3 = 36$	Architect	Ongoing	Validate significant tech choices with a small spike before committing team-wide.
2	$3 \times 4 \times 2 = 24$	Lead Developer	Ongoing	Enforce linting and code review before merge; light refactor pass each iteration.
2	$3 \times 2 \times 2 = 12$	QA / Test Lead	Ongoing	Set a minimum coverage target for core workflows; include testing time in sprint planning.
3	$3 \times 2 \times 3 = 18$	DevOps / Integration Lead	Each sprint	Integrate and deploy continuously each iteration (CI pipeline + staging) rather than only at the end.
2	$3 \times 3 \times 2 = 18$	Configuration Lead	Each sprint	Work together as a team to make sure each developer has the same environment, so that there aren't accidental bugs added due to environment issues. Concretely: everyone runs the stack through the committed docker compose file; CI builds the same backend and frontend images that Render and Vercel deploy; every environment variable is documented in .env.example with a dev-only default.

3	$3*2*3=18$	Team Lead	Before Iteration 2 demo	Document the deploy path end to end in the README so it can be followed by someone else; give a second member access to the Render, Vercel and staging-mirror settings; pair on the next pipeline change so the knowledge is shared rather than explained.
2	$3*3*2=18$	Configuration Lead	Each sprint	Keep every hostname in configuration rather than hard-coded, so a move needs no code change (CORS_ORIGINS, APP_BASE_URL, VITE_API_BASE are already environment variables). Warm the staging instance before a demo rather than relying on a cold start. Record the option of buying one domain and pointing app./api. subdomains at both hosts — SDD §6.3 Option B — as the permanent fix if the team judges it worth the cost.

1	2*2*1=4	Security Lead	2026-09-21	Rate-limit the route per IP and count successful creations rather than failures, because here the successes are the abuse. Cover it with integration tests that prove the cap, prove successes are counted, and prove the authentication limiter's own budget is untouched.
2	4*2*2=16	Security Lead	Before Iteration 2 demo	Create the insert-only Atlas role defined in code/docs/audit-db-role.md (find and insert on auditevents; full access granted collection by collection on the other six, because MongoDB privileges are additive and a database-wide readWrite cannot be carved back out), then reassign the API user to it. A staged test already exists and runs once AUDIT_ROLE_TEST_URI points at a real authenticated deployment.

The team member who is adding the new feature should be the one who determines what is correct for the new functionality that was added.

Work together as a team to make sure each developer has the same environment. This is useful so that there aren't accidental bugs added due to environment issues.

Execution summary	Status
No one dropped the class so far	Open
No new members to date.	Open
All members active to date.	Open
No duplicate work to date.	Open
No occurrences to date.	Open
No occurrences to date.	Open
Config to be added in Sprint 1.	Open
No occurrences to date.	Open
No occurrences to date.	Open

No occurrences to date.	Open
No occurrences to date.	Open
No occurrences to date.	Open
No occurrences to date.	Open
No occurrences to date.	Open
No occurrences to date.	Open
Test framework to be selected in Sprint 1.	Open
No occurrences to date.	Open
No occurrences to date.	Open
Initial data model under review.	Open

No occurrences to date.	Open
No occurrences to date.	Open
No occurrences to date.	Open
CI/CD pipeline to be set up in Sprint 1.	Open
Docker Compose, the committed .env. example and the CI image build are all in place. The known remaining gap is the database: local and CI run an ephemeral MongoDB, staging runs Atlas free tier, and production is not yet provisioned.	Open

Identified during
Iteration 1: the staging
deploy pushes main to
a personal mirror
repository (Alexm-
picard/SafeDrop) that
Render and Vercel
watch, and the
production deploy job
is still a placeholder.
Both currently depend
on one member's
accounts.

Iteration 1 runs
entirely on free tiers
with platform-assigned
hostnames. No outage
so far; no mitigation
beyond configuration
has been needed yet.

Open

Open

Closed. Capped at 5 per IP per hour (RATE_LIMIT_ORG_CREATE_MAX, default 5) with skipSuccessfulRequests set to false. Merged in PR #43; three integration tests cover it.

Open — blocked on Atlas console access, which no team member has used for role changes yet. The role definition, the migration and the staged test merged in PR #44; until the role is applied, SR-8 rests on the application layer alone.

Closed

Open

